

浙江理工大学科技与艺术学院

KEYI COLLEGE OF ZHEJIANG SCI-TECH UNIVERSITY

# 程序设计基础课程设计

# 课程设计报告

课设名称 ：腾讯新闻热点数据获取及可视化展示

课设时间：2025/12/19

专　　业　大数据管理与应用

班　　级　大数据管理与应用 1

学生姓名　杨济同

学　　号　Xc24590104

指导老师　叶杰锋

# 一、需求分析

本次我爬取的是腾讯新闻网站的热点区块内容，并最终实现对爬取的数据进行 web 展示可视化处理，具体内容包括：爬取站点，腾讯新闻首页（https://news.qq.com）；抓取新闻标题、新闻链接、新闻图片（URL 及本地存储）、新闻摘要、抓取时间等信息；最终目的完成从 “页面数据爬取→数据存储（CSV+MongoDB）→Web 页面可视化展示” 的全流程实现，验证网络爬虫、数据存储与 Web 开发的技术结合能力。

# 二、设计思路

**开发环境：**Python 3.8+、Windows/macOS/Linux 操作系统；开发工具：PyCharm（代码编写）、Chrome 浏览器（页面模拟）、MongoDB（数据库存储）；使用类库：Selenium（模拟浏览器爬取）、lxml（页面解析）、Flask（Web 服务）、Bootstrap（前端样式）、pymongo（MongoDB 连接）等。

**核心思路与抓取流程：**环境依赖安装 → Selenium 启动浏览器加载目标页面 → 等待页面元素加载完成 → 解析页面中的新闻标题/链接/图片等元素 → 数据存储至 CSV 文件与 MongoDB → Flask 读取存储数据 → 结合 Bootstrap 模板实现 Web 可视化展示。

**流程图：**

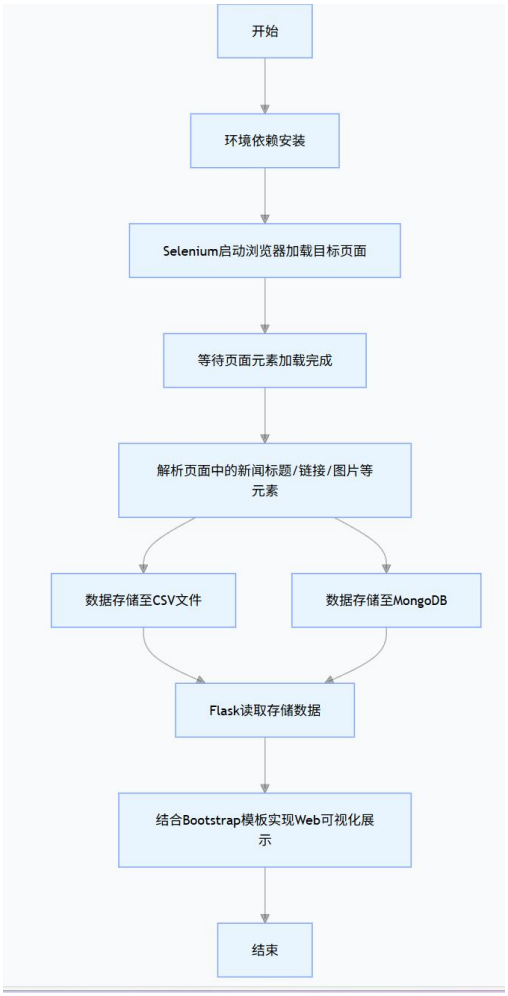



图 2-1(流程图)

# 三、详细设计

## 1. 导入库及查看网站对应源码（准备工作）

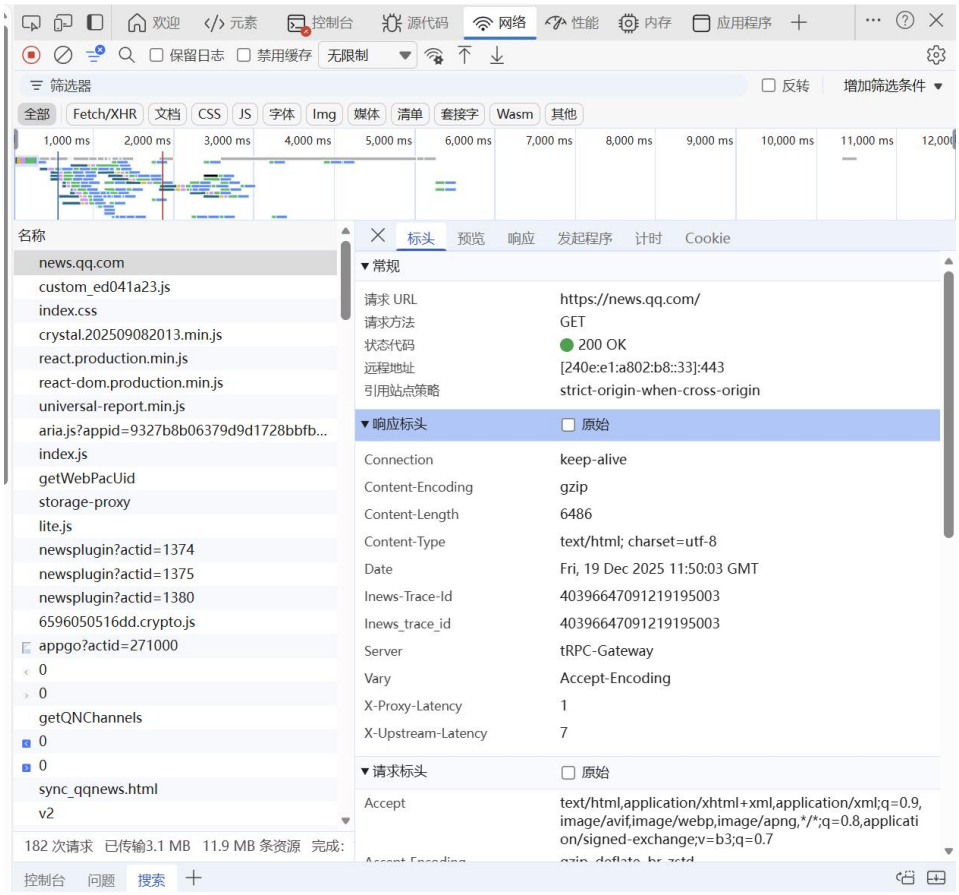



图 3-1（网站网络请求）

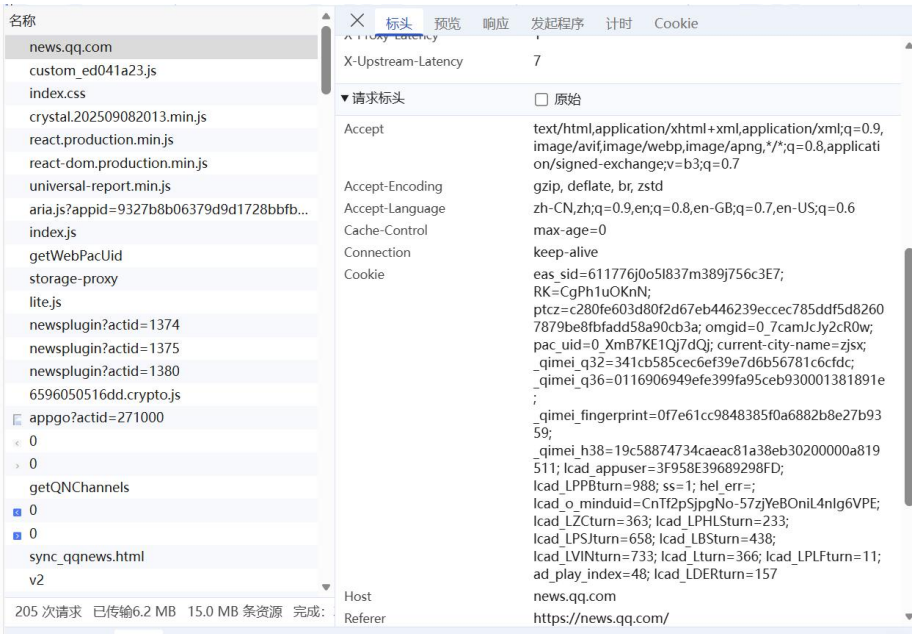



图 3-2（请求标头）

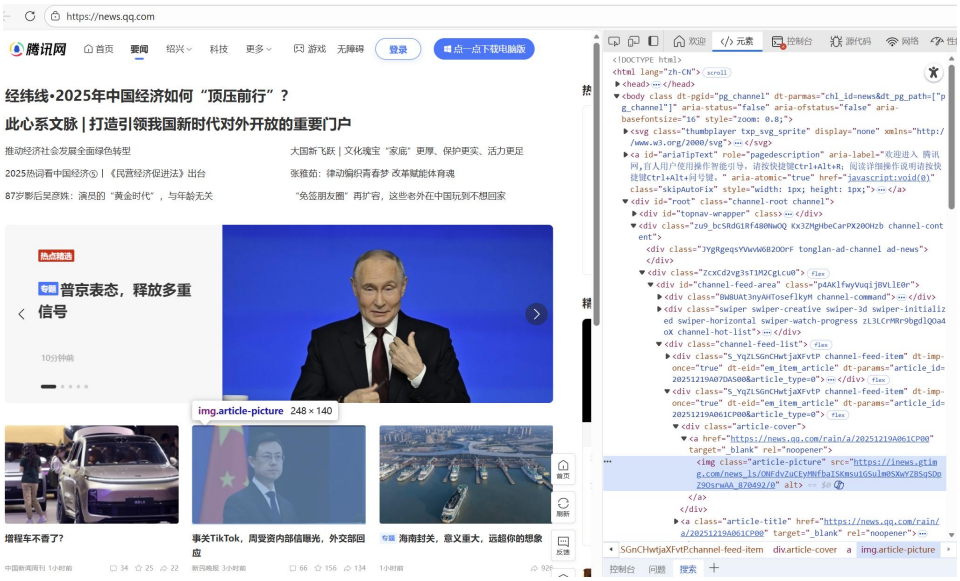



图 3-3（元素标签）

```python
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from webdriver_manager.chrome import ChromeDriverManager
from lxml import etree
import csv
import time
from pymongo import MongoClient
import requests
import os
```

图 3-4（导入函数）

导入必要的依赖库及模块组并查找网站对应热点区块的 div，class，id 等源码标签，请求，标头便于后期查找对应位置和叠层关系

## 2. 初始化定义

```python
# -------------------------- 配置项 --------------------------
TENCENT_NEWS_URL = "https://news.qq.com"
CSV_SAVE_PATH = "腾讯新闻爬取结果.csv"
MONGO_URI = "mongodb://localhost:27017/"  # 本地MongoDB默认地址，不用改
MONGO_DB_NAME = "tencent_news_db"
MONGO_COLLECTION_NAME = "tencent_news_list"
IMAGE_SAVE_DIR = "tencent_news_images"
DOWNLOAD_IMAGES = True  # 想下载图片就改成True
```

图 3-5（初始化定义）

图 3-5 为爬虫项目的全局配置项初始化，通过定义固定变量统一管理核心参数。既明确了爬取目标站点、数据存储介质（CSV+MongoDB）的路径 / 名称，也通过开关控

制图片下载功能，便于后续功能的灵活调整；同时 MongoDB 的连接信息采用默认本地地址，保证了开发环境的兼容性。

```
# 创建图片保存目录
if DOWNLOAD_IMAGES and not os.path.exists(IMAGE_SAVE_DIR):
    os.makedirs(IMAGE_SAVE_DIR)
```

图 3-6（保存图片）

图 3-6 实现图片本地存储目录的“按需创建”逻辑：首先判断 DOWNLOAD_IMAGES 开关是否开启（仅在需要下载图片时执行），再通过 os.path.exists 检查目标目录是否已存在，避免重复创建目录导致的系统报错；最终通过 os.makedirs 创建目录，用于统一存放后续爬取的新闻图片资源，保证本地文件的有序管理。

### 3. 抓取数据

```
def crawl_tencent_news():  1 个用法
    print("开始加载腾讯新闻首页...")
    driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()))
    try:
        driver.get(TENCENT_NEWS_URL)
        # 等待核心新闻列表加载
        WebDriverWait(driver, timeout: 10).until(
            EC.presence_of_element_located((By.XPATH, '//div[contains(@class, "channel-feed-item")]'))
        )
        time.sleep(3)  # 等待图片资源加载
        html = etree.HTML(driver.page_source)
        news_list = []
```

图 3-7（抓取函数）

爬虫核心函数的初始化逻辑，通过 “浏览器驱动初始化→页面加载→元素等待→页面解析” 的流程，完成从 “模拟访问站点” 到 “获取可解析数据源” 的过渡。其中结合了显式等待 + 强制等待的组合策略，既保证了页面核心元素的加载完整性，也覆盖了图片等异步资源的加载场景；同时通过 etree.HTML 转换页面源码，为后续的元素提取提供了技术基础。

```
# 匹配当前页面的新闻条目
news_items = html.xpath('//div[contains(@class, "channel-feed-item")]')
if not news_items:
    print("❌ 未找到新闻条目")
    return []
```

图 3-8（入口筛选）

新闻数据解析的“入口筛选”逻辑，通过 XPath 模糊匹配页面中承载新闻内容的容器元素（channel-feed-item 类 div），既适配了页面元素 class 的动态变化特性，也通

过容错判断确保：若页面结构变更导致无法匹配到新闻条目时，程序会及时终止并提示，避免后续解析步骤因空数据出现崩溃。

## 4. 解析数据

```
# 解析每个新闻条目
for idx, item in enumerate(news_items, 1):
    # 1. 提取标题
    title = item.xpath('.//span[contains(@class, "hot-article-title-text")]/text()')
    if not title:
        title = item.xpath('.//span[contains(@class, "question-title-text")]/text()')
    if not title:
        title = item.xpath('.//span[contains(@class, "article-title-text")]/text()')
    title = title[0].strip() if title else "无标题"

    # 2. 提取新闻链接【关键修改1：空链接替换为"无链接"】
    link = item.xpath('.//a[contains(@class, "article-base-info")]/@href')
    if not link:
        link = item.xpath('.//a[contains(@class, "question-title")]/@href')
    if not link:
        link = item.xpath('.//div[contains(@class, "article-cover")]/a/@href')
    link = link[0].strip() if link else "无链接"  # 空链接不再留空，填"无链接"

    # 3. 提取图片URL
    img_url = item.xpath('.//img[contains(@class, "article-image")]/@src')
    if not img_url:
        img_url = item.xpath('.//img[contains(@class, "article-picture")]/@src')
    if not img_url:
        img_url = item.xpath('.//div[contains(@class, "answer-desc")]//img/@src')
    img_url = img_url[0].strip() if img_url else "无图片"
```

图 3-9（定位标签）



```
        img_url = img_url[0].strip() if img_url else "无图片"

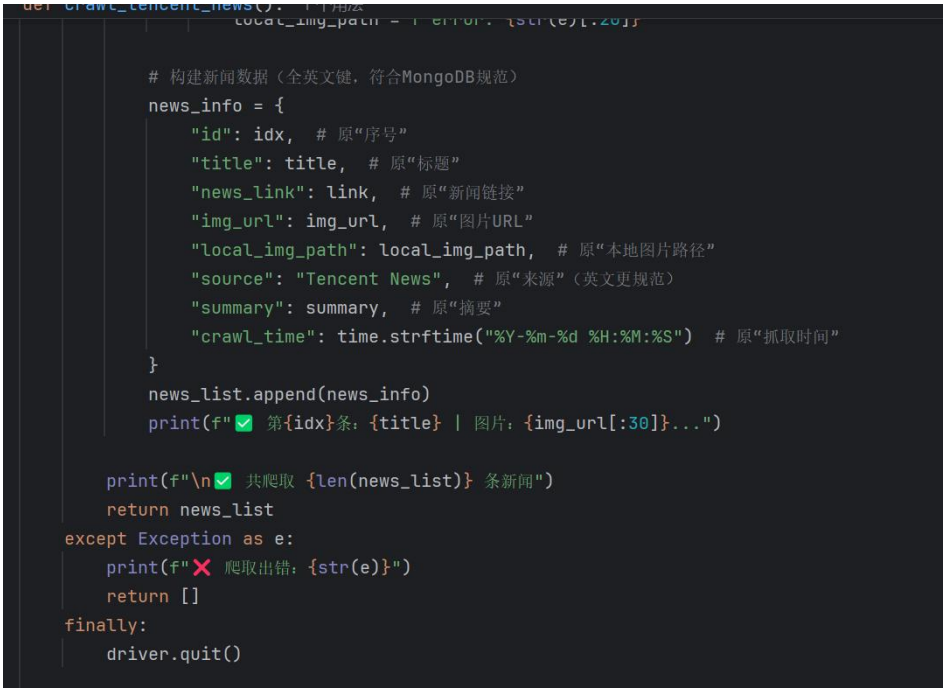
        # 4. 提取摘要
        summary = item.xpath('.//div[contains(@class, "answer-desc")]/text()')
        if not summary:
            summary = item.xpath('.//div[contains(@class, "hot-article-desc")]/text()')
        summary = ''.join([s.strip() for s in summary]) if summary else title

        # 5. 下载图片（可选）
        local_img_path = "未启用下载"
        if DOWNLOAD_IMAGES and img_url != "无图片":
            try:
                img_filename = f"news_img_{idx}_{int(time.time())}.jpg"
                local_img_path = os.path.join(IMAGE_SAVE_DIR, img_filename)
                response = requests.get(img_url, timeout=10, stream=True)
                if response.status_code == 200:
                    with open(local_img_path, 'wb') as f:
                        for chunk in response.iter_content(chunk_size=1024):
                            if chunk:
                                f.write(chunk)
                else:
                    local_img_path = "下载失败"
            except Exception as e:
                local_img_path = f"异常: {str(e)[:20]}"

        # 构建新闻数据
        news_info = {
            "序号": idx,
            "标题": title,
```

图 3-10（下载图片）

新闻数据的核心解析逻辑，针对腾讯新闻页面 “多模块、多 class” 的特点，采用 “多备选 XPath + 容错赋值” 的策略：标题 / 链接 / 图片均设计了 3 种备选匹配规则，覆盖热点新闻、问答新闻、普通新闻等不同模块的元素结构；每个字段均添加了 “空值替换” 处理（如空标题→“无标题”），避免因个别新闻的元素缺失导致程序报错，同时保证后续存储的数据结构统一；所有文本 / 链接均通过 strip()去除首尾空格，提升数据的整洁性。图片下载部分通过 “条件判断 + 异常捕获 + 分片写入” 实现：仅在满足下载条件时执行，同时对请求失败、网络异常等情况做了容错标记，保证单张图片下载失败不影响整体爬虫流程；



```
            # 构建新闻数据（全英文键，符合MongoDB规范）
            news_info = {
                "id": idx,  # 原"序号"
                "title": title,  # 原"标题"
                "news_link": link,  # 原"新闻链接"
                "img_url": img_url,  # 原"图片URL"
                "local_img_path": local_img_path,  # 原"本地图片路径"
                "source": "Tencent News",  # 原"来源"（英文更规范）
                "summary": summary,  # 原"摘要"
                "crawl_time": time.strftime("%Y-%m-%d %H:%M:%S")  # 原"抓取时间"
            }
            news_list.append(news_info)
            print(f"✅ 第{idx}条：{title} | 图片：{img_url[:30]}...")

        print(f"\n✅ 共爬取 {len(news_list)} 条新闻")
        return news_list
    except Exception as e:
        print(f"❌ 爬取出错：{str(e)}")
        return []
    finally:
        driver.quit()
```

图 3-11（构建结构化数据）

完成 “单条数据结构化→数据聚合→函数收尾” 的流程：采用全英文键构建字典，既符合 MongoDB 等数据库的命名规范，也避免了中文键在跨系统存储时的编码问题；通过 news_list.append 实现数据聚合，将分散的单条新闻整合为统一列表，为后续批量存储提供基础；日志打印分为 “单条进度 + 总量统计”，便于调试时实时跟踪爬取状态；try-except-finally 结构实现了 “异常容错 + 资源释放”：异常时返回空列表避免后续错误，finally 中强制关闭浏览器驱动，彻底解决爬虫后浏览器进程残留的问题。

## 5. 存储数据

```
def save_to_csv(news_list):  1 个用法
    """保存到CSV（英文表头）"""
    if not news_list:
        print("❌ 无数据保存到CSV")
        return
    # 全英文表头
    fieldnames = ["id", "title", "news_link", "img_url", "local_img_path", "source", "summary", "crawl_time"]
    with open(CSV_SAVE_PATH, "w", encoding="utf-8-sig", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(news_list)
    print(f"✅ CSV已保存到：{CSV_SAVE_PATH}")
```

图 3-12（存储 csv）

实现了 MongoDB 存储的 “连接管理 + 去重前置筛选”：用 client.admin.command('ping')做连接校验，提前暴露连接问题；采用 “投影查询（仅查 title 和 news_link）” 减少数据传输量，提升查询效率；以 “标题 + 链接” 为唯一标识去

重，既保证数据不冗余，也避免同一新闻因爬取时间不同而重复入库。

```python
def save_to_mongodb(news_list):  1 个用法
    """保存到MongoDB（全英文键）"""
    if not news_list:
        print("❌ 无数据保存到MongoDB")
        return
    try:
        client = MongoClient(MONGO_URI)
        client.admin.command('ping')  # 测试连接
        db = client[MONGO_DB_NAME]
        col = db[MONGO_COLLECTION_NAME]

        # 去重逻辑（英文键匹配）
        existing = set()
        for item in col.find( *args: {}, {"title": 1, "news_link": 1}):
            title = item.get("title", "no_title")
            news_link = item.get("news_link", "no_link")
            existing.add((title, news_link))

        new_news = []
        for n in news_list:
            key = (n["title"], n["news_link"])
            if key not in existing:
                new_news.append(n)

        if new_news:
            col.insert_many(new_news)
            print(f"✅ MongoDB新增 {len(new_news)} 条数据")
```

图 3-13（存储 mongodb）

```python
def save_to_mongodb(news_list):  1个用法
        col = db[MONGO_COLLECTION_NAME]

        # 去重逻辑（英文键匹配）
        existing = set()
        for item in col.find( *args: {}, {"title": 1, "news_link": 1}):
            title = item.get("title", "no_title")
            news_link = item.get("news_link", "no_link")
            existing.add((title, news_link))

        new_news = []
        for n in news_list:
            key = (n["title"], n["news_link"])
            if key not in existing:
                new_news.append(n)

        if new_news:
            col.insert_many(new_news)
            print(f"✅ MongoDB新增 {len(new_news)} 条数据")
        else:
            print("✅ 无新增数据（已去重）")
        client.close()
    except Exception as e:
        print(f"❌ MongoDB保存失败：{str(e)}")
        print("💡 排查建议：1. 检查MongoDB是否启动 2. 确认本地27017端口未被占用")
```

图 3-14（存储 mongodb）

MongoDB 存储的 “写入执行 + 异常容错 + 资源释放” 环节：批量插入（insert_many）是高并发场景下的性能优化手段，相比单条插入能减少 90% 以上的网络交互开销；try-except 捕获所有异常并给出排查建议，避免程序崩溃且降低调试难度；finally 中强制关闭连接，彻底解决“连接未释放导致的资源泄露”问题。

```python
if __name__ == "__main__":
    print("=" * 50)
    print("    腾讯新闻爬取工具（修复MongoDB版）")
    print("=" * 50)
    print(f"📌 图片下载：{'已启用' if DOWNLOAD_IMAGES else '未启用'}")
    if DOWNLOAD_IMAGES:
        print(f"📌 图片目录：{os.path.abspath(IMAGE_SAVE_DIR)}")
    print("-" * 50)

    news_data = crawl_tencent_news()
    if news_data:
        save_to_csv(news_data)
        save_to_mongodb(news_data)
    print("\n✅ 操作完成！")
```

图 3-15（运行主函数）

```python
from flask import Flask, render_template
import csv
import os

# 初始化Flask应用
app = Flask(__name__,
            static_folder="tencent_news_images",  # 图片保存目录
            template_folder="templates")

# 配置项（和爬虫保持一致）
CSV_SAVE_PATH = "腾讯新闻爬取结果.csv"
```

图 3-16（导入 flask）

web 端服务通过 flask 驱动，配置静态文件夹 tencent_news_images（存储本地图片）与模板文件夹 templates（存放前端页面）

```python
def read_news_data():  1 个用法
    """读取CSV中的新闻数据（修复图片显示）"""
    news_data = []
    if not os.path.exists(CSV_SAVE_PATH):
        return {"data": [], "count": 0}

    # 读取CSV文件
    with open(CSV_SAVE_PATH, "r", encoding="utf-8-sig") as f:
        reader = csv.DictReader(f)
        for row in reader:
            # 修复图片路径逻辑
            img_src = "无图片"
            # 优先用本地下载的图片
            if row["本地图片路径"] not in ["未启用下载", "无", "下载失败"] and os.path.exists(row["本地图片路径"]):
                img_filename = os.path.basename(row["本地图片路径"])
                img_src = f"/{img_filename}"
            # 无本地图片则用在线URL（补全https协议）
            elif row["图片URL"] != "无图片":
                img_src = row["图片URL"]
                if img_src.startswith("//"):
                    img_src = "https:" + img_src

            # 组装新闻数据
            news_data.append({
```

图 3-17（定义读取函数）

```python
def read_news_data():  1个用法
    with open(CSV_SAVE_PATH, "r", encoding="utf-8-sig") as f:
            if row["本地图片路径"] not in ["未启用下载", "无", "下载失败"] and os.path.exists(row["本地图片路径"]):
                img_filename = os.path.basename(row["本地图片路径"])
                img_src = f"/{img_filename}"
            # 无本地图片则用在线URL（补全https协议）
            elif row["图片URL"] != "无图片":
                img_src = row["图片URL"]
                if img_src.startswith("//"):
                    img_src = "https:" + img_src

            # 组装新闻数据
            news_data.append({
                "id": row["序号"],
                "title": row["标题"],
                "link": row["新闻链接"] if row["新闻链接"] != "无链接" else "#",
                "img": img_src,
                "summary": row["摘要"],
                "source": row["来源"],
                "crawl_time": row["抓取时间"]
            })
    return {"data": news_data, "count": len(news_data)}


@app.route('/')
```

图 3-18（定义读取函数）

定义 read_news_data 函数，读取 CSV 中的新闻数据，新闻图片优先采用本地已下载的文件，若未找到，则采取第二种方法以线上 url 获取为主

```python
def index():
    """首页展示新闻数据"""
    news_info = read_news_data()
    return render_template( template_name_or_list: "index.html",
                           news_list=news_info["data"],
                           total_count=news_info["count"])
```

图 3-19（获取数据）

定义 index 函数，引用文件夹 templates 中的 index.html 样式，数据由 news_list 和 total_count 两个变量组成，搭建一个前端 web 服务器

```html
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>腾讯新闻爬取结果展示</title>
    <!-- 引入Bootstrap样式 -->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
    <style>
        .news-card {transition: transform 0.2s; margin-bottom: 20px;}
        .news-card:hover {transform: translateY(-5px); box-shadow: 0 8px 16px rgba(0,0,0,0.1);}
        .news-img {height: 200px; object-fit: cover;}
        .header {background-color: #0d6efd; color: white; padding: 20px 0; margin-bottom: 30px;}
        .stats {background-color: #f8f9fa; padding: 15px; border-radius: 8px; margin-bottom: 20px;}
    </style>
</head>
<body>
    <!-- 页面头部 -->
    <div class="header text-center">
        <div class="container">
            <h1>腾讯新闻爬取结果</h1>
            <p class="lead">共爬取 {{ total_count }} 条新闻数据</p>
        </div>
    </div>
```

图 3-20（网页源码）

```
<html lang="zh-CN">
<head>
    </style>
</head>
<body>
    <!-- 页面头部 -->
    <div class="header text-center">
        <div class="container">
            <h1>腾讯新闻爬取结果</h1>
            <p class="lead">共爬取 {{ total_count }} 条新闻数据</p>
        </div>
    </div>

    <!-- 主要内容 -->
    <div class="container">
        <!-- 数据统计 -->
        <div class="stats">
            <div class="row">
                <div class="col-md-4">
                    <h5>📊 总新闻数</h5>
                    <p class="h3">{{ total_count }}</p>
                </div>
                <div class="col-md-4">
                    <h5>🖼️ 含图片新闻数</h5>
                    <p class="h3">{{ news_list | selectattr('img', '!=', '无图片') | list | length }}</p>
```

图 3-21（网页源码）

```
<html lang="zh-CN">
<body>
    <div class="container">

        <!-- 无数据提示 -->
        {% if total_count == 0 %}
        <div class="text-center mt-5">
            <h3>暂无爬取数据</h3>
            <p class="text-muted">请先运行爬虫脚本生成CSV数据</p>
        </div>
        {% endif %}
    </div>

    <!-- 底部 -->
    <footer class="mt-5 py-3 bg-light text-center">
        <div class="container">
            <p>新闻数据爬取展示系统 | 更新时间：{{ news_list[0].crawl_time if news_list else '暂无' }}</p>
        </div>
    </footer>

    <!-- 引入Bootstrap脚本 -->
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

图 3-22（网页源码）

前端 index.html 文件代码，通过 Jinja2 模板引擎，将数据渲染到 Bootstrap 风格的前端页面中，实现响应式展示，页面所有动态数据均来自后端读取的 CSV 文件，实现“数据与视图分离”，通过 Bootstrap 的 col-md-4 col-sm-6 实现 “中等屏幕一行 3 个卡片、小屏幕一行 2 个卡片、手机端一行 1 个卡片”，适配不同设备；区分在线图片（以 http 开头）和本地图片（通过 url_for('static', ...)拼接 Flask 静态文件路径）；添加 onerror 事件，当图片加载失败时自动替换为占位图，避免页面出现 “裂图”；selectattr('img', '!=', '无图片')：筛选出包含图片的新闻；list | length：统计筛选后的新闻数量；新闻卡片添加 hover 动画（上浮 + 阴影），提升用户体验；新闻标题添加 text-truncate

超长省略 +title 属性，兼顾美观和完整信息展示。

## 6. 运行及展示




图 3-23（pycharm 结果）




图 3-24（pycharm 结果）

pycharm 端爬取数据展示

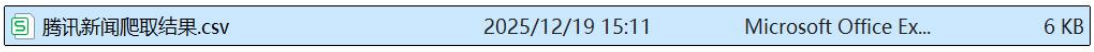



图 3-25（本地存储）




图 3-26（本地存储）

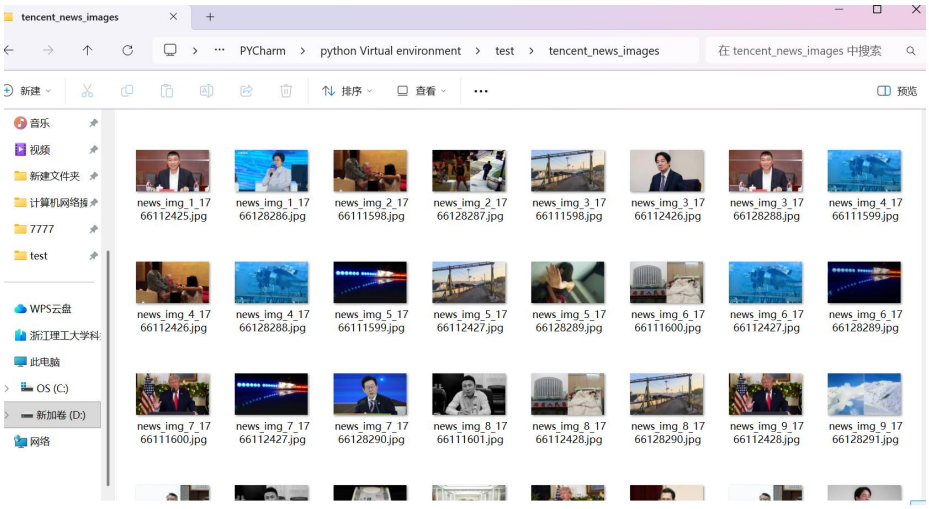



图 3-27（本地存储）

本地 csv 文件及存储图片数据集

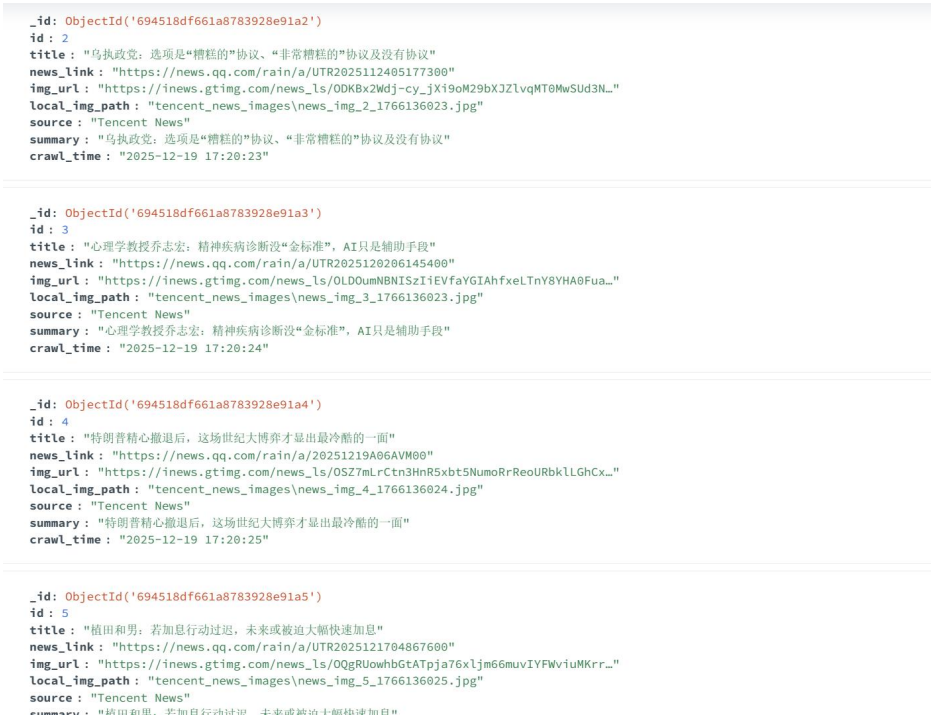



图 3-28（存储 mongodb）



图 3-29（存储 mongodb）

MongDB 端爬取数据展示

```
"D:\PYCharm\python Virtual environment\test\.venv\Scripts\python.exe" "D:\PYCharm\python Virtual environment\test\app.py"
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.50.51:5000
Press CTRL+C to quit
```

图 3-30（web 服务启动）

调用 flask 函数在本地搭建 web 服务器，ip 地址为：192.168.50.51:5000

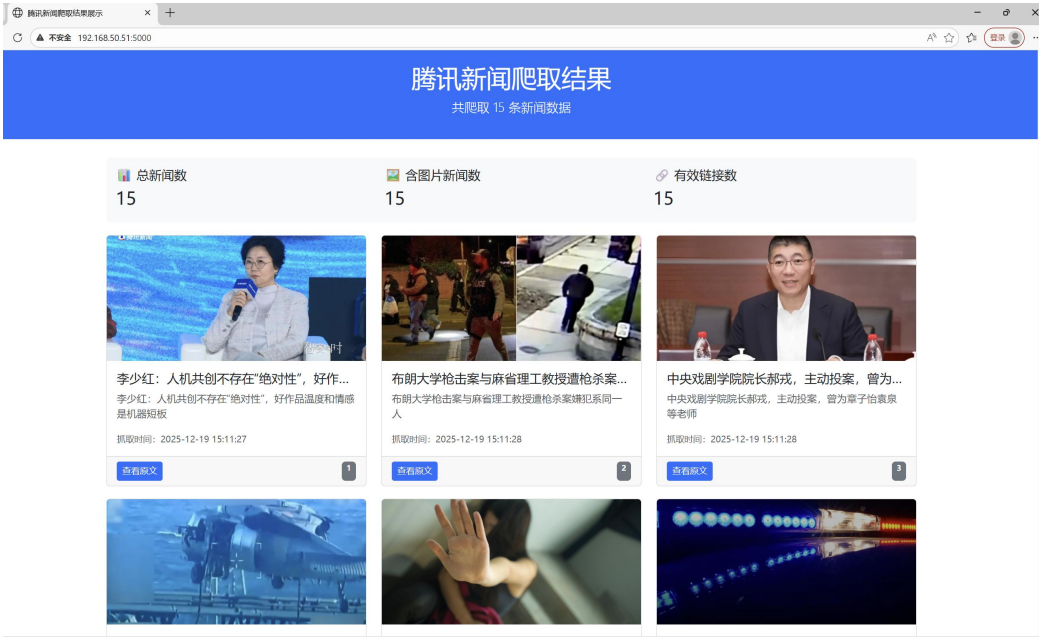



图 3-31（网页展示）

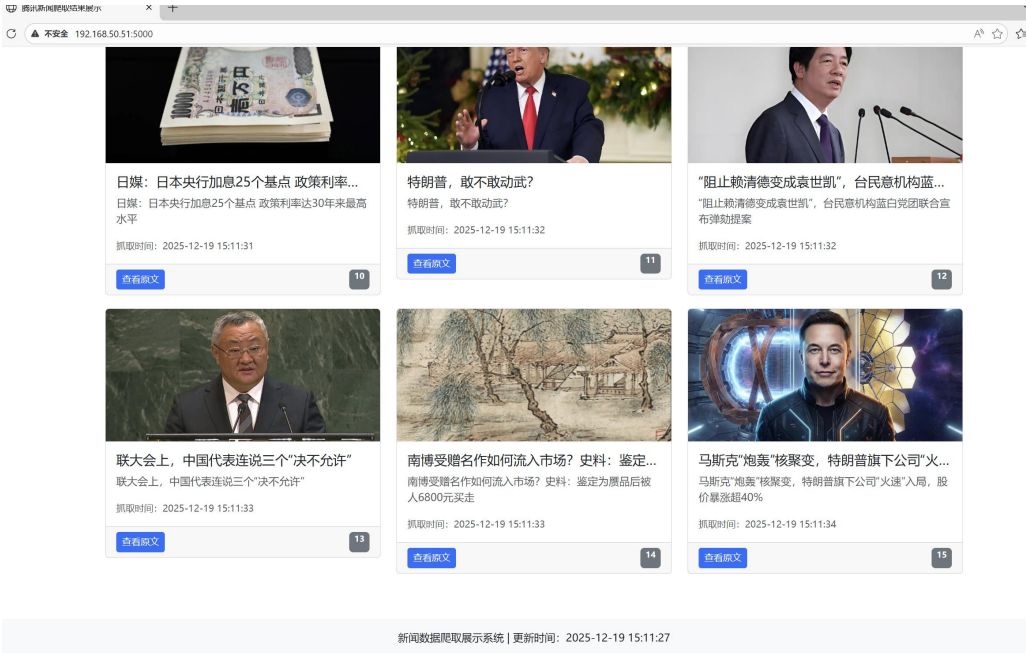


图 3-32（网页展示）

Web 端口服务将本地数据可视化展示输出

## 四、心得体会

本次项目中遇到的核心问题及解决方法：MongoDB 保存失败（提示“新闻链接”异常）；解决方法：将空新闻链接替换为“无链接”占位符，优化去重逻辑的字段匹配；Web 展示时图片不显示；解决方法：补全在线图片的 https:协议前缀、修正本地图片的 Flask 静态文件路径，增加图片加载失败的占位图；

通过本次课设，不仅掌握了 Selenium 模拟爬取、数据多格式存储的技术，也理解了 Web 服务与前端模板结合的可视化逻辑，同时提升了问题排查与代码调试的能力。

## 五、核心代码

```
def crawl_tencent_news():
    driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()))
    try:
        driver.get("https://news.qq.com")
        WebDriverWait(driver, 10).until(
            EC.presence_of_element_located((By.XPATH, '//div[contains(@class, "channel-feed-item")]'))
        )
        html = etree.HTML(driver.page_source)
        news_items = html.xpath('//div[contains(@class, "channel-feed-item")]')
        # 后续解析与数据封装逻辑...
    finally:
        driver.quit()
```

图 5-1

## 爬虫核心代码注释：

webdriver.Chrome(...)：自动下载并启动 Chrome 驱动，模拟真实浏览器访问页面（解决静态爬取无法获取动态加载内容的问题）；WebDriverWait(...)：显式等待 10 秒，直到页面中 “新闻条目容器” 加载完成，避免因页面未加载完导致的元素解析失败；

**痛点：**腾讯新闻首页是动态渲染页面，直接爬取 HTML 无法获取完整内容，需模拟浏览器加载。

```
# 2. 页面解析（XPath提取新闻元素）
html = etree.HTML(driver.page_source)
news_items = html.xpath('//div[contains(@class, "channel-feed-item")]')
for idx, item in enumerate(news_items, 1):
    # 提取标题
    title = item.xpath('.//span[contains(@class, "hot-article-title-text")]/text()')
    title = title[0].strip() if title else "无标题"
    # 提取图片URL
    img_url = item.xpath('.//img[contains(@class, "article-image")]/@src')
    img_url = img_url[0].strip() if img_url else "无图片"
```

图 5-2

etree.HTML(driver.page_source)：将浏览器加载后的完整页面源码转为可解析的 HTML 对象；XPath 语法：通过“类名模糊匹配”提取元素（如 contains(@class, "channel-feed-item")），适配页面元素类名的细微变化；容错处理：提取标题 / 图片时添加 if title else "无标题"，避免因个别元素缺失导致程序崩溃。

```
# 3. 图片本地下载
if DOWNLOAD_IMAGES and img_url != "无图片":
    img_filename = f"news_img_{idx}_{int(time.time())}.jpg"
    local_img_path = os.path.join(IMAGE_SAVE_DIR, img_filename)
    response = requests.get(img_url, timeout=10, stream=True)
    if response.status_code == 200:
        with open(local_img_path, 'wb') as f:
            for chunk in response.iter_content(chunk_size=1024):
                f.write(chunk)
```

图 5-3

requests.get(img_url, stream=True)：流式下载图片，避免一次性加载大图片占用过多内存；chunk_size=1024：按 1KB 分片写入文件，提升大图片下载的稳定性；命名规则：news_img_序号_时间戳.jpg，避免图片文件名重复覆盖。

```
def save_to_csv(news_list):
    fieldnames = ["序号", "标题", "新闻链接", "图片URL", "本地图片路径", "来源", "摘要", "抓取时间"]
    with open(CSV_SAVE_PATH, "w", encoding="utf-8-sig", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()  # 写入表头
        writer.writerows(news_list)  # 批量写入数据
```

图 5-4

```
def save_to_mongodb(news_list):
    client = MongoClient(MONGO_URI)
    client.admin.command('ping')  # 测试MongoDB连接
    db = client[MONGO_DB_NAME]
    col = db[MONGO_COLLECTION_NAME]
    # 去重逻辑：按“标题+链接”去重
    existing = set((item["标题"], item["新闻链接"]) for item in col.find({}, {"标题":1, "新闻链接":1}))
    new_news = [n for n in news_list if (n["标题"], n["新闻链接"]) not in existing]
    if new_news:
        col.insert_many(new_news)
```

图 5-5

## CSV 数据存储核心代码：

client.admin.command('ping')：检测 MongoDB 服务是否启动，提前暴露连接问题；去重逻辑：通过集合（set）存储已有的“标题 + 链接”组合，避免重复数据入库；insert_many：批量插入数据，提升写入效率（对比逐条 insert_one）

```python
# 1. Flask应用初始化
app = Flask(__name__,
            static_folder="tencent_news_images",  # 本地图片静态目录
            template_folder="templates")  # HTML模板目录

# 2. 读取CSV数据并处理图片路径
def read_news_data():
    with open(CSV_SAVE_PATH, "r", encoding="utf-8-sig") as f:
        reader = csv.DictReader(f)
        for row in reader:
            # 补全在线图片协议（解决//xxx.com/xx.jpg无法加载的问题）
            if row["图片URL"] != "无图片" and row["图片URL"].startswith("//"):
                row["图片URL"] = "https:" + row["图片URL"]
            # 本地图片路径拼接（适配Flask静态文件规则）
            if os.path.exists(row["本地图片路径"]):
                img_src = f"/{os.path.basename(row['本地图片路径'])}"
```

图 5-6

## WEB 端核心代码：

static_folder：指定本地图片的存储目录，Flask 会自动将该目录下的文件映射为/文件名的访问路径；图片路径修复：补全以//开头的图片 URL 协议（浏览器无法识别无协议的 URL），保证在线图片正常加载。

**指导老师评价：**

| 项目评价 | 优 | 良 | 中 | 一般 | 较差 |
|---|---|---|---|---|---|
| 课堂纪律：按时上课，无迟到早退 | | | | | |
| 完成态度：认真完成项目设计，勤奋好学，积极思考 | | | | | |
| 项目完成度：按照要求完成项目设计 | | | | | |
| 课程质量：代码正确，按时完成，界面美观 | | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 课程总结：思路清晰，调理清楚，全面总结了心得体会和不足 | | | | | |
| **总评成绩** | **签名：**<br>**时间：** | | | | |